Justificación (documentación) de la arquitectura de software de Prosta-AI

Yeison Bolaños Ruiz

Juan David Riascos Erazo

Sergio Gabriel Meneses Silva

Yimis Buenaventura

Fernando José Angulo Núñez

Facultad de Ingeniería, Universidad Santiago de Cali

Programa de Ingeniería de Sistemas

Gestión de Proyectos de TI

Jair Enrique Sanclemente Castro

24 de octubre de 2026

Justificación (documentación) de la arquitectura de software de Prosta-AI

1. Identificación e información general

Este documento justifica las decisiones arquitectónicas de Prosta-AI y organiza su documentación según las prácticas que recomiendan el estándar IEEE 1471 y el enfoque «Vistas y más allá» del SEI, descritos por Gómez (s. f.). Complementa el documento «Arquitectura de software de Prosta-AI» (Equipo Prosta-AI, 2026a), que contiene las figuras, tablas y flujos, y los referencia sin repetirlos. La Tabla 1 resume la identificación del documento.

Tabla 1
Identificación del documento

Elemento	Contenido
Sistema	Prosta-AI: aplicación web académica para cargar estudios de imagen desidentificados, segmentar la próstata, visualizar la ROI y obtener métricas cuando hay referencia. No da diagnósticos ni recomendaciones clínicas.
Alcance del documento	Nivel arquitectónico. Los esquemas de tablas se detallan en la arquitectura de datos; las rutas, los algoritmos y los protocolos de prueba quedan para el diseño técnico.
Ubicación en el proyecto	Paquete 1.3.2.4 de la EDT; actividades 1306 a 1309 del Ciclo 2 (26 al 29 de octubre de 2026).
Documentos relacionados	SRS v1.0, Plan de Gestión de Calidad v0.1, cronograma Gantt del 11/10/2026, arquitectura de software y arquitectura del sistema del 22/10/2026, y arquitectura de datos del 24/10/2026 (Equipo Prosta-AI, 2026a, 2026b, 2026c, 2026d, 2026e).
Decisión que se justifica	Arquitectura cliente-servidor por capas, con React en la presentación y un backend modular en FastAPI y Python; PostgreSQL y almacenamiento privado; inferencia con PyTorch y MONAI; despliegue mínimo viable de un solo proceso de backend.

2. Marco de referencia

2.1 Qué es la arquitectura y por qué se justifica

Bass et al. (2013), retomados por Pressman y Maxim (2020), definen la arquitectura como las estructuras fundamentales de un sistema: sus elementos, las relaciones entre ellos y las propiedades necesarias para entenderlo. Le asignan tres funciones: facilitar la comunicación entre

las partes interesadas, dejar registradas las primeras decisiones de diseño y ser una abstracción transferible y reutilizable. Esas tres funciones son la razón de ser de este documento.

En SWEBOK v4.0 (IEEE Computer Society, 2024) la arquitectura es un área de conocimiento nueva, separada del diseño, porque se la reconoce como una disciplina propia. Según la descripción del área en el resumen de SEBoK (Washizaki et al., s. f.), la arquitectura busca «satisfacer lo suficiente» a todos los interesados, mientras que el diseño se ocupa de transformar esa visión en una implementación factible. Esta distinción ordena el trabajo: este documento justifica la visión (qué estructura y por qué); el diseño técnico del Ciclo 2 en adelante la convertirá en implementación. El editor del área de arquitectura de SWEBOK v4.0, Rich Hilliard, es también el referente que Gómez (s. f.) cita para ISO/IEC 42010/IEEE 1471.

2.2 Enfoques de documentación considerados

La Tabla 2 compara los enfoques revisados y su uso en Prosta-AI.

Tabla 2
Enfoques de documentación de arquitectura considerados

Enfoque	Idea central	Uso en Prosta-AI
Kruchten, 4+1 vistas (1995)	Vistas lógica, de procesos, de desarrollo y física, articuladas por escenarios de casos de uso.	Correspondencia para comprobar que no falte ningún aspecto (sección 4.3).
Nord et al. (1995)	Vistas conceptual, de módulos, de ejecución y de código.	Contraste; la vista de código se difiere.
SEI, «Vistas y más allá»	Tipos de vista (módulo, conectores y componentes, asignación); selección de vistas por interesados; paquetes de vista; estilos y guía de estilos.	Guía el procedimiento de selección de vistas (sección 4) y la definición del estilo (sección 6).
IEEE 1471	Descripción organizada por puntos de vista según los intereses de los interesados; exige justificación y registro de inconsistencias.	Guía la estructura de este documento (secciones 3, 5, 6 y 8).

Gómez (s. f.) señala que la tendencia actual es documentar solo las vistas útiles para los interesados, en lugar de ajustarse a un número fijo. Prosta-AI sigue esa tendencia: documenta tres vistas más los escenarios, no cinco.

3. Partes interesadas e intereses

El estándar IEEE 1471 considera incompleta una descripción si algún interés no se atiende en al menos una vista. Por eso se parte de los interesados reales del proyecto, tomados del Plan de Gestión de Calidad y del SRS (Tabla 3).

Tabla 3
Partes interesadas e intereses principales en la arquitectura

Interesado	Interés principal
Propietario (docente)	Que el diseño cumpla el alcance académico, no se confunda con software médico y permita evaluar el ciclo de gestión del proyecto.
Director	Planificar el trabajo de los siete ciclos, controlar versiones y presentar evidencias.
Analista	Trazabilidad entre requisitos y componentes; monitoreo y control.
Diseñador	Que el flujo de carga a ROI sea comprensible y que el aviso experimental esté en toda pantalla de resultados.
Desarrollador	Estructura clara de módulos, stack definido y factibilidad de cómputo.
Tester	Poder verificar privacidad, flujo completo y rendimiento en cada integración.
Usuario académico	Cargar un estudio, ver la ROI y entender sus límites sin formación médica ni de programación.

4. Selección de vistas

Se sigue el procedimiento de tres pasos del SEI que describe Gómez (s. f.): listar vistas candidatas por interesado, combinarlas y priorizarlas.

4.1 Vistas candidatas y nivel de información requerido

La Tabla 4 muestra el nivel de información que cada interesado requiere de cada vista. Los niveles son una valoración del equipo.

Tabla 4
Vistas candidatas por interesado

Interesado	**Lógica** (módulos y componentes)	**Ejecución** (flujo y procesos)	**Despliegue** (asignación)	**Código** (repositorio)	**Escenarios** (casos de uso)
Propietario	G	G	G	N	D
Director	G	G	G	G	G
Analista	D	G	N	G	D
Diseñador	D	G	N	N	D
Desarrollador	D	D	D	D	G
Tester	G	D	D	G	D
Usuario académico	N	N	N	N	D

Nota. D = requerido a detalle; G = de manera general; N = ninguno.

4.2 Combinación y priorización

- **Vista lógica de componentes (prioridad 1).** Combina la vista de módulos y la de conectores y componentes del SEI: muestra los siete componentes, sus responsabilidades y el orden lógico del flujo. Es la que más interesados requieren a detalle. Corresponde a la Figura 1 y la Tabla 2 del documento de arquitectura.
- **Escenarios de casos de uso (prioridad 1).** Los casos CU01 a CU06 relacionan la arquitectura con los requisitos y con las demás vistas, como el «+1» de Kruchten. Corresponde a la Tabla 3 del documento de arquitectura.
- **Vista de despliegue mínimo viable (prioridad 2).** Es la vista de asignación del SEI: relaciona los componentes con el servidor, PostgreSQL y el almacenamiento privado. Corresponde a la Figura 2 del documento de arquitectura.
- **Vista de ejecución.** Se combina con la lógica: el orden del flujo (admisión, trabajo pendiente, tarea local secuencial, resultado) se describe en la sección «Flujo de procesamiento» y no justifica una vista aparte, porque el mínimo viable no tiene concurrencia compleja.
- **Vista de código (prioridad 3, diferida).** La organización del repositorio es detalle de

diseño técnico y de construcción; se documentará con el diseño del Ciclo 2 en adelante y con el requisito RA-01.

4.3 Correspondencia con el modelo 4+1

La Tabla 5 relaciona cada vista de Kruchten (1995) con su equivalente en Prosta-AI.

Tabla 5
Correspondencia con el modelo 4+1 de Kruchten

Vista de Kruchten	Equivalente en Prosta-AI
Lógica	Vista lógica de componentes (Figura 1 y Tabla 2 del documento de arquitectura).
Procesos	Flujo de procesamiento y tarea local secuencial.
Desarrollo	Módulos separados dentro de un único backend; detalle diferido al diseño técnico.
Física	Despliegue mínimo viable (Figura 2 del documento de arquitectura).
Escenarios	Casos de uso CU01 a CU06 (Tabla 3 del documento de arquitectura).

5. Puntos de vista

Cada punto de vista declara, como pide IEEE 1471, a quién sirve, qué intereses satisface, con qué técnica se modela, con qué método se analiza y por qué se eligió (Tabla 6).

Tabla 6
Puntos de vista de la arquitectura de Prosta-AI

Punto de vista	Interesados	Intereses que satisface	Técnica de modelado	Método analítico	Justificación
Lógico de componentes	Analista, diseñador, desarrollador, director	Cobertura funcional, trazabilidad, mantenibilidad	Diagrama de bloques y tabla de responsabilidades	Revisión de trazabilidad requisito-componente (sección 9)	Es la referencia común del equipo y permite detectar requisitos sin componente.
Despliegue	Desarrollador, tester, propietario	Rendimiento, factibilidad de cómputo, privacidad del almacenamiento	Diagrama de bloques de nodos	Medición de CPU, GPU, memoria y tiempos en el Ciclo 3 (actividad 1408)	Atiende el riesgo 1 del Acta y la meta de 120 s.
Escenarios	Todos	Que el flujo completo sea comprensible y verificable	Tabla de casos de uso con requisitos relacionados	Recorrido de cada caso contra la matriz de trazabilidad	Relaciona las vistas entre sí y valida la arquitectura.

6. Justificación del estilo arquitectónico

6.1 Estilo elegido

Shaw y Garlan (1996) definen un estilo como una descripción de elementos, conectores, topología y restricciones sobre la interacción. La Tabla 7 lo aplica a Prosta-AI.

Tabla 7
Componentes del estilo aplicados a Prosta-AI

Componente del estilo	**Prosta-AI**
Elementos	Siete componentes: presentación React, API y servicios de aplicación, validación y preparación, segmentación y ROI, evaluación, repositorios y archivos, orientación opcional.
Conectores	Solicitudes de la interfaz a la API; acceso a datos mediante repositorios; tabla de trabajos en PostgreSQL como punto de coordinación entre la API y la tarea local.
Topología	Cliente-servidor por capas: presentación, aplicación, procesamiento de imagen y persistencia.
Restricciones	La API no llama directamente a la segmentación; el procesamiento toma los trabajos de uno en uno; el módulo de IA permanece desacoplado de la API; el acceso se limita a alias y contraseña cifrada, sin roles ni permisos adicionales.

La U-Net 3D es el modelo neuronal; no es el estilo arquitectónico de la aplicación. Confundirlos llevaría a justificar la estructura del sistema con argumentos sobre el modelo.

6.2 Comparación de alternativas por atributos

Siguiendo la idea de razonar sobre atributos de calidad por estilo (Klein et al., 1999), la Tabla 8 compara cada alternativa con los criterios que importan al proyecto. La valoración es cualitativa y del equipo; las mediciones del Ciclo 3 la confirmarán o corregirán.

Tabla 8
Comparación de alternativas arquitectónicas

Criterio	Monolito modular (elegida)	Backend con ejecutor separado	Microservicios
Costo de integración y despliegue en el Ciclo 6	Bajo	Medio	Alto
Aislamiento de la carga de inferencia	Bajo: compite con la API por CPU y memoria	Alto	Alto
Coordinación y recuperación de fallos	Simple	Compleja	Muy compleja
Ajuste al alcance académico y a recursos gratuitos	Alto	Medio	Bajo
Cumplimiento de RC-02 (módulo de IA desacoplado)	Se cumple por separación de módulos y tabla de trabajos	Se cumple	Se cumple
Posibilidad de evolucionar si las mediciones lo exigen	Alta: el código ya está separado en módulos	No aplica	No aplica

6.3 Por qué se elige el monolito modular

1. **Es suficiente para el alcance.** El flujo completo debe integrarse en el Ciclo 6 con cinco integrantes y recursos de cómputo limitados al equipo personal y a servicios gratuitos como Google Colab (SRS, sección 1.6). Una arquitectura más compleja consumiría tiempo que el cronograma no tiene.
2. **No renuncia a la separación.** Aunque se despliega un solo proceso, el código se mantiene separado en módulos y la API solo crea trabajos pendientes. Esto cumple RC-02 y deja abierta la salida hacia un ejecutor separado.
3. **Evita complejidad sin evidencia.** Las estructuras de concurrencia o recuperación solo se construirán si hay datos que demuestren que hacen falta, en línea con la advertencia de Pressman y Maxim (2020) de decidir mientras los cambios aún son fáciles.
4. **Es reversible.** La decisión se revisa si las mediciones muestran que la interfaz se bloquea o que los tiempos no alcanzan el objetivo de rendimiento (sección 10).

Los microservicios se descartan porque su complejidad operativa es excesiva para un proyecto

académico y no resuelven ningún riesgo identificado en el Acta.

7. Justificación por atributos de calidad

En SWEBOK v4.0 la seguridad aparece como un área de conocimiento propia y la arquitectura como la disciplina que debe asegurar los niveles de calidad desde el inicio (IEEE Computer Society, 2024). Prosta-AI traduce sus atributos prioritarios en decisiones y en evidencia verificable (Tabla 9).

Tabla 9

Atributos de calidad, decisiones y evidencia

Atributo	Decisión arquitectónica	Requisitos del SRS	Evidencia prevista
Privacidad	Admisión y validación detallada de cada archivo; eliminación de metadatos identificables o rechazo; originales, máscaras y pesos en almacenamiento privado; usuarios con solo alias y contraseña cifrada, verificada dos veces al crearla	RF-03, RF-04, RF-05, RB-01, RB-02, RC-03	Inspección de privacidad antes de cada integración (métrica P06)
Rendimiento	Procesamiento local secuencial con medición separada de preparación, inferencia y recorrido completo	RR-01 (≤ 120 s)	Medición temprana en el Ciclo 3 y protocolo de rendimiento del Ciclo 6 (métrica P04)
Trazabilidad	Cada resultado se relaciona con la versión del modelo y la procedencia del estudio; estados de trabajo registrados	RF-18, RF-19, RA-02	Matriz de trazabilidad (métrica P01)
Mantenibilidad	Módulos separados, stack definido, responsabilidades explícitas por componente	RC-01, RC-02, RA-01	Revisión entre pares y revisión de arquitectura
Usabilidad	Flujo en pasos distintos (cargar, procesar con el botón «Procesar», visualizar) con estado visible y aviso experimental	RF-07, RF-13 a RF-17, RU-01	Prueba de usabilidad con compañeros del curso
Disponibilidad de la API	La API solo admite y registra; el trabajo pesado corre en una tarea aparte	RF-08	Comprobación de que la interfaz consulta el estado mientras corre la tarea local
Calidad de la segmentación	Evaluación con Dice e IoU sobre un conjunto de prueba separado por paciente	RF-23, RF-24, RC-04	Métricas P03 y P07

8. Consistencia entre vistas

IEEE 1471 exige registrar las inconsistencias entre vistas y un procedimiento para mantenerlas coherentes. A continuación se anotan las comprobaciones hechas, las decisiones de diseño que precisan el SRS y las diferencias que permanecen.

8.1 Comprobaciones superadas

Tabla 10
Comprobaciones de consistencia superadas

Comprobación	Resultado
La Figura 1 y la Tabla 2 del documento de arquitectura describen los mismos siete componentes	Coherente
Figura 2: el backend único contiene la API, la tarea local, la validación, la inferencia y la evaluación; PostgreSQL y el almacenamiento privado equivalen al componente 6	Coherente
Los casos de uso CU01 a CU06 coinciden con el diagrama de casos de uso y la Tabla 2 del SRS	Coherente
Las actividades 1306 a 1309 (Ciclo 2), 1408 (Ciclo 3) y 1701 a 1706 (Ciclo 6) coinciden con el cronograma	Coherente
RC-02 (módulo de IA desacoplado) frente a un solo proceso de backend	Coherente: el desacople es lógico (módulos y tabla de trabajos), no de proceso

8.2 Decisiones de diseño y diferencias registradas

La Tabla 11 recoge las dos decisiones de diseño que precisan el SRS y la única diferencia que permanece abierta.

Tabla 11
Decisiones de diseño y diferencias registradas

ID	Tipo	Descripción	Relación con el SRS
DEC-01	Decisión de diseño	Primer acceso: si el nombre de usuario no existe, el sistema ofrece crearlo y pide escribir la contraseña dos veces. Si ambas coinciden, guarda el usuario con la contraseña cifrada.	Es el acceso básico (RF-01, requisito 4.7): solo alias y contraseña cifrada, sin perfiles, roles, permisos ni datos personales. Cumple RC-05 y RB-02.
DEC-02	Decisión de diseño	Inicio del procesamiento: el usuario carga el estudio (CU02) y luego pulsa «Procesar» (CU03). Desde ese momento, validación, preparación, segmentación y evaluación se ejecutan solas.	RF-06 y el requisito 4.2 se leen como automatización completa desde la acción «Procesar», sin pasos manuales intermedios.

ID	Tipo	Descripción	Relación con el SRS
INC-03	Diferencia abierta	El SRS programa casi todas las pruebas funcionales en el Ciclo 6, mientras el plan de calidad exige verificar el flujo completo y la privacidad en cada integración.	Adelantar los controles de integración a los ciclos 2 y 3, aprovechando la medición temprana de la actividad 1408.

8.3 Procedimiento de mantenimiento de la consistencia

- Al cerrar cada ciclo, el analista revisa por muestreo requisito, componente, prueba y entregable en la matriz de trazabilidad.
- Todo cambio de componente, flujo o despliegue se refleja en las tres vistas y en la matriz antes de integrarse.
- Las diferencias nuevas se agregan a la Tabla 11 con su resolución.

9. Trazabilidad entre requisitos y componentes

La Tabla 12 presenta la asignación propuesta por el equipo de los requisitos del SRS a los componentes del documento de arquitectura. Cada requisito tiene al menos un componente responsable.

Tabla 12
Asignación de requisitos del SRS a componentes

Componente	Requisitos del SRS que cubre
1. Presentación React	RF-07, RF-13, RF-14, RF-15, RF-17, RU-01
2. API y servicios de aplicación	RF-01, RF-02, RF-03, RF-04, RF-06, RF-08, RF-19, RC-05
3. Validación y preparación	RF-04, RF-05, RF-09, RF-10, RR-01 (tiempo de preparación)
4. Segmentación y ROI	RF-11, RF-12, RR-01 (tiempo de inferencia)
5. Evaluación	RF-23, RF-24, RC-04
6. Repositorios y archivos	RF-18, RB-01, RB-02, RC-03
7. Orientación opcional	RF-20, RF-21, RF-22
Transversales	RF-16, RN-01 a RN-03, RC-01, RC-02, RC-06, RA-01, RA-02

10. Condiciones de revisión de la decisión

La justificación se sostiene sobre supuestos que la propia arquitectura declara y que se confirman con evidencia, no por afirmación (Tabla 13).

Tabla 13
Condiciones de revisión de la decisión arquitectónica

Condición	Qué se observa	Decisión que habilita
Tiempo de inferencia	Medición en el Ciclo 3 con un volumen representativo, pesos reales y el hardware previsto	Si supera 120 s: Google Colab, un modelo preentrenado más ligero o menor resolución; si aun así no se cumple, documentar la limitación
Bloqueo de la interfaz	Consulta de estado mientras corre la tarea local	Si se bloquea: separar el procesador en una etapa posterior
Privacidad e integridad	Rechazo de datos no aptos y recuperación del resultado permitido	Ajustar validación, aislamiento y trazabilidad sin ampliar el alcance funcional
Datos y recursos	Límites de tamaño, conjunto de datos y disponibilidad de GPU	Ajustar el diseño técnico del Ciclo 2 y actualizar el registro de riesgos

Supuestos declarados: estudios públicos desidentificados, acceso con nombre de usuario y contraseña sin roles ni datos personales, formatos DICOM y NIfTI, y un modelo de segmentación 3D que funcione con los recursos del equipo.

11. Cumplimiento de las prácticas de documentación

La Tabla 14 indica dónde se cumple cada práctica recomendada por Gómez (s. f.), IEEE 1471 y el SEI.

Tabla 14
Cumplimiento de las prácticas de documentación

Práctica recomendada	Dónde se cumple
Identificación e información general	Sección 1
Identificar interesados y sus intereses	Sección 3
Documentar solo las vistas útiles para los interesados	Sección 4
Puntos de vista con técnica, método analítico y justificación	Sección 5

Práctica recomendada	Dónde se cumple
UML no es el único lenguaje posible	Se usan diagramas de bloques y tablas junto con casos de uso
Justificación del tipo de arquitectura y de los puntos de vista	Secciones 4 y 6
Atributos de calidad con análisis	Sección 7
Consistencia entre vistas	Sección 8
Matriz de trazabilidad entre requisitos y elementos de la arquitectura	Sección 9
Antecedentes, supuestos y resultados de análisis (SEI)	Secciones 6, 7 y 10
Documento bajo línea base	Se versionará en el repositorio con las etiquetas de ciclo del plan de calidad

Conclusión

La arquitectura cliente-servidor por capas con backend modular responde a lo que el proyecto necesita ahora: integrar el flujo completo en el Ciclo 6, proteger la privacidad de los estudios, mantener la trazabilidad de cada resultado y conservar la posibilidad de separar el procesamiento si las mediciones lo exigen. La documentación con tres vistas y escenarios satisface los intereses de los siete tipos de interesados, y el registro de decisiones deja constancia de cómo el primer acceso y el inicio del procesamiento se ajustan al SRS.

Referencias

Bass, L., Clements, P., & Kazman, R. (2013). *Software architecture in practice* (3rd ed.). Addison-Wesley.

Clements, P., Bachmann, F., Bass, L., Garlan, D., Ivers, J., Little, R., Nord, R., & Stafford, J. (2002). *Documenting software architectures: Views and beyond*. Addison-Wesley.

Equipo Prosta-AI. (2026a). *Arquitectura de software de Prosta-AI* [Documento interno]. Universidad Santiago de Cali.

Equipo Prosta-AI. (2026b). *Especificación de requisitos de software (SRS) de Prosta-AI* (Versión 1.0) [Documento interno]. Universidad Santiago de Cali.

Equipo Prosta-AI. (2026c). *Plan de Gestión de Calidad de Prosta-AI* (Versión 0.1) [Documento interno]. Universidad Santiago de Cali.

Equipo Prosta-AI. (2026d). *Arquitectura del sistema de Prosta-AI* [Documento interno]. Universidad Santiago de Cali.

Equipo Prosta-AI. (2026e). *Arquitectura de datos de Prosta-AI* [Documento interno]. Universidad Santiago de Cali.

Gómez, O. (s. f.). *Documentando la arquitectura de software: Principios básicos*. OSGG. http://www.osgg.net

IEEE Architecture Working Group. (2000). *IEEE Std 1471-2000: Recommended practice for*

architectural description of software-intensive systems. IEEE.

IEEE Computer Society. (2024). *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0* (H. Washizaki, Ed.). https://www.computer.org/education/bodies-of-knowledge/software-engineering

Klein, M., Kazman, R., & Nord, R. (1999). *Attribute-based architectural styles* (CMU/SEI-99-TR-22). Software Engineering Institute, Carnegie Mellon University.

Kruchten, P. (1995). The 4+1 view model of architecture. *IEEE Software, 12*(6), 42–50.

Nord, R. L., Soni, D., & Hofmeister, C. (1995). Software architecture in industrial applications. *Proceedings of the 17th International Conference on Software Engineering (ICSE '95)*, 196–207. ACM.

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Shaw, M., & Garlan, D. (1996). *Software architecture: Perspectives on an emerging discipline*. Prentice Hall.

Washizaki, H., Sanchez-Segura, M.-I., Garbajosa, J., Tockey, S., Nidiffer, K. E., & Reilly, A. D. (s. f.). *An overview of the SWEBOK Guide*. SEBoK. https://sebokwiki.org/wiki/An_Overview_of_the_SWEBOK_Guide